

# Projeto 2 — DGEMM Paralela com MPI

**Autores:** Klaus Almeida e Adrielle Maria Alves Queiroz

**Curso:** Bacharelado em Ciência da Computação, UESC

**Disciplina:** DEC107 — Processamento Paralelo

## 1. Introdução

O objetivo deste projeto é implementar e analisar a multiplicação geral de matrizes densas (DGEMM) em um ambiente de memória distribuída utilizando o padrão MPI e comparar seu desempenho com as versões sequencial e paralela em OpenMP desenvolvidas no Projeto 1. A multiplicação de duas matrizes quadradas ( $A^{(nn)}$ ) e ( $B^{(nn)}$ ) resulta em uma matriz ( $C^{(nn)}$ ) e é uma operação fundamental em computação científica, aprendizagem de máquina e simulação numérica. O algoritmo tradicional possui complexidade cúbica ( $n^3$ ), de modo que seu tempo de execução cresce rapidamente com o tamanho da matriz.

Paralelizar a DGEMM é essencial para torná-la viável em matrizes maiores. Em arquiteturas de memória compartilhada, o OpenMP distribui o trabalho entre threads; em ambientes de memória distribuída, vários processos comunicam-se por troca de mensagens. O Projeto 2 enfatiza o uso do MPI para decompor a matriz em blocos de linhas ou colunas e explorar comunicação ponto-a-ponto e coletivas. Também inclui testes de corretude e análise de precisão numérica, ausentes no Projeto 1.

## 2. Metodologia

### 2.1 Implementações

**Versão sequencial.** A versão sequencial utiliza três laços for aninhados: o laço externo itera sobre as linhas ( $i$ ) de ( $C$ ), o laço intermediário percorre as colunas ( $j$ ) e o laço interno acumula o produto escalar entre a linha ( $i$ ) de ( $A$ ) e a coluna ( $j$ ) de ( $B$ ). As matrizes são armazenadas como vetores unidimensionais (C-style arrays) conforme a especificação do projeto. Para melhorar o uso de cache, a versão sequencial revisada transpõe a matriz ( $B$ ) uma única vez para ( $B^T$ ) e executa a multiplicação ( $A B^T$ ) com acesso contíguo à memória.

**Versão OpenMP.** O paralelismo em memória compartilhada foi obtido aplicando a diretiva `#pragma omp parallel for` ao laço mais externo. Essa diretiva cria uma equipe de threads e distribui as linhas de ( $C$ ) entre elas, evitando condições de corrida e garantindo bom balanceamento de carga. Foram testadas configurações de 2, 4, 8 e 12 threads.

**Versão MPI.** Para a implementação MPI, adotou-se a decomposição por blocos de linhas: o processo de rank 0 gera ( $A$ ) e ( $B$ ), depois distribui blocos de linhas de ( $A$ ) para os demais processos com `MPI_Scatterv` e transmite ( $B$ ) inteira para todos com `MPI_Bcast`. Cada processo calcula seu bloco de ( $C$ ) multiplicando o bloco de ( $A$ ) pelo vetor transposto de ( $B$ ) ( $B^T$ ) localmente. Os resultados são reunidos no rank 0 através de `MPI_Gatherv`. Foi implementado um validador que recalcula ( $C$ ) no rank 0 e mede a

diferença relativa máxima entre a saída MPI e a sequencial, aceitando erro menor que  $(10^{-9})$ .

## 2.2 Hardware e ambiente de teste

- **Processador:** AMD Ryzen 5 7600X, 6 núcleos (12 threads).
- **Memória RAM:** 16 GB.
- **Placa de vídeo:** NVIDIA GeForce RTX 4070 SUPER.
- **Sistema operacional:** Microsoft Windows 11 Pro (versão 10.0.26200) com driver Game Ready 581.57 (14/10/2025).
- **Ambiente de execução:** Ubuntu 24.04.3 LTS rodando no Subsistema do Windows para Linux (WSL 2). A máquina virtual WSL foi configurada para 12 processadores lógicos, 7 723 MB de memória e 2 048 MB de arquivo de troca.
- **Compiladores e bibliotecas:** g++ com flags -O3 -march=native; OpenMP versão 4.0; OpenMPI 4.1.5 com a opção --oversubscribe para permitir múltiplos processos.

## 2.3 Procedimentos de medição

Para cada combinação de tamanho (N) e número de processos (p) ou threads, foram executadas 7 rodadas a fim de reduzir a variabilidade estatística. As matrizes (A) e (B) foram preenchidas com números pseudoaleatórios de ponto flutuante utilizando a semente 42 para garantir reprodutibilidade. Em cada rodada:

1. As matrizes (A) e (B) são alocadas e preenchidas.
2. Registra-se o tempo inicial com `std::chrono::high_resolution_clock::now()`.
3. Executa-se a multiplicação de matrizes (sequencial, OpenMP ou MPI).
4. O tempo final é registrado e calcula-se o intervalo.
5. Para as versões paralelas, realiza-se uma rodada de aquecimento previamente para reduzir efeitos de cache.

Ao término das 7 execuções calcula-se a média e o desvio-padrão dos tempos. Para o cálculo de *speedup* ( $Sp$ ) utiliza-se a média da versão sequencial como referência: ( $Sp = T/Tp$ ). A eficiência é dada por ( $Ep = Sp/p$ ). A rotina de validação computa a diferença relativa máxima entre a saída MPI e a sequencial, conforme sugerido no enunciado.

## 2.4 Métricas de desempenho

- **Tempo de execução médio:** média aritmética dos tempos medidos em 7 rodadas.
- **Desvio-padrão:** medida da dispersão temporal.
- **Speedup:** razão entre o tempo sequencial e o tempo paralelo.
- **Eficiência:** speedup dividido pelo número de processos ou threads.
- **Diferenciação algorítmica:** no MPI, o caso ( $p = 1$ ) já incorpora a otimização da transposição de (B) e portanto apresenta speedup maior que 1 sem paralelismo.

## 3. Resultados

### 3.1 Teste de corretude

Todos os valores de diferença relativa máxima entre as matrizes resultantes da versão MPI e da versão sequencial foram inferiores a ( $10^{-12}$ ), confirmando que a implementação MPI produz resultados corretos.

### 3.2 Tabelas de resultados

*Tabela 1 – Versão Sequencial*

| N    | média (s)  | desvio-padrão (s) |
|------|------------|-------------------|
| 512  | 0.323085   | 0.022770          |
| 1024 | 3.276246   | 0.278315          |
| 2048 | 38.330613  | 0.183269          |
| 4096 | 357.691905 | 1.229622          |

*Tabela 2 – Versão OpenMP (com speedup e eficiência)*

| N    | threads | média (s)       | desvio (s) | speedup      | eficiência |
|------|---------|-----------------|------------|--------------|------------|
| 512  | 2       | 0.136397        | 0.003433   | 2.37         | 1.18       |
| 512  | 4       | 0.073241        | 0.005958   | 4.41         | 1.10       |
| 512  | 8       | 0.043702        | 0.000888   | 7.39         | 0.92       |
| 512  | 12      | <b>0.030747</b> | 0.000796   | <b>10.51</b> | 0.88       |
| 1024 | 2       | 1.283190        | 0.040094   | 2.55         | 1.27       |

| N    | threads | média (s)        | desvio (s) | speedup      | eficiência |
|------|---------|------------------|------------|--------------|------------|
| 1024 | 4       | 0.726404         | 0.030375   | 4.51         | 1.13       |
| 1024 | 8       | 0.408499         | 0.017063   | 8.03         | 1.00       |
| 1024 | 12      | <b>0.335720</b>  | 0.005049   | <b>9.76</b>  | 0.81       |
| 2048 | 2       | 17.890750        | 0.125971   | 2.14         | 1.07       |
| 2048 | 4       | 9.003334         | 0.079328   | 4.26         | 1.06       |
| 2048 | 8       | 4.417554         | 0.049024   | 8.67         | 1.08       |
| 2048 | 12      | <b>3.329061</b>  | 0.009894   | <b>11.52</b> | 0.96       |
| 4096 | 2       | 202.934174       | 0.465738   | 1.76         | 0.88       |
| 4096 | 4       | 101.082689       | 0.466358   | 3.54         | 0.88       |
| 4096 | 8       | 55.486867        | 0.195210   | 6.45         | 0.81       |
| 4096 | 12      | <b>46.108461</b> | 0.081044   | <b>7.76</b>  | 0.65       |

*Tabela 3 – Versão MPI (com speedup e eficiência)*

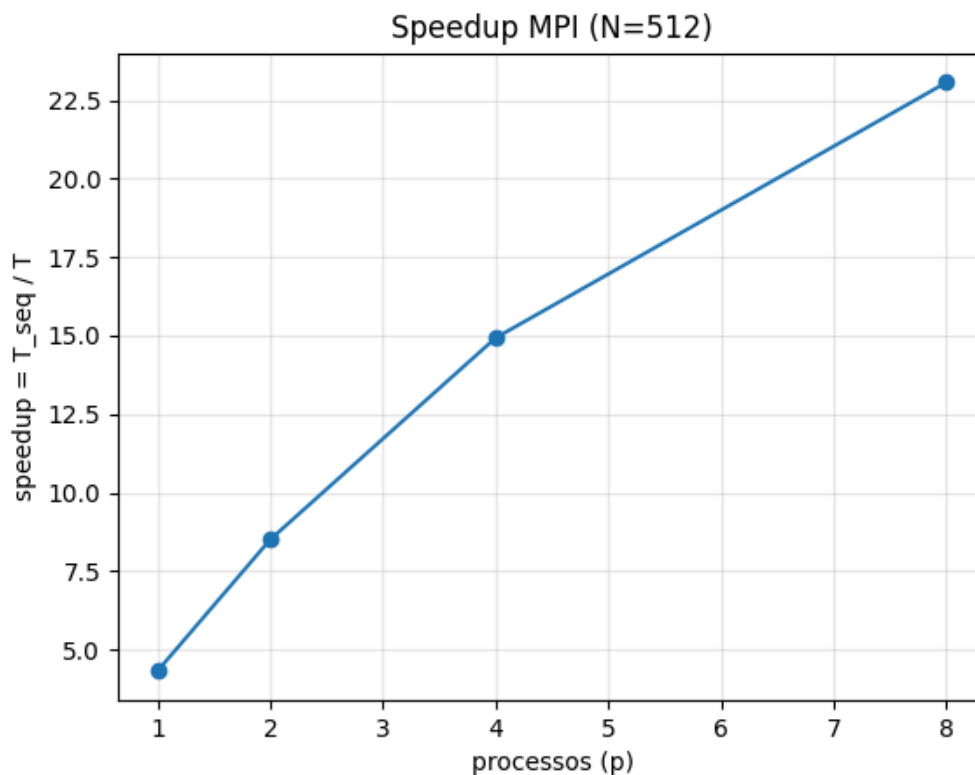
| N    | procs | média (s)       | desvio (s) | speedup      | eficiência |
|------|-------|-----------------|------------|--------------|------------|
| 512  | 1     | 0.074747        | 0.003313   | 4.32         | 4.32       |
| 512  | 2     | 0.037979        | 0.001072   | 8.51         | 4.25       |
| 512  | 4     | 0.021641        | 0.001314   | 14.93        | 3.73       |
| 512  | 8     | <b>0.014007</b> | 0.000681   | <b>23.07</b> | 2.88       |
| 1024 | 1     | 0.610556        | 0.021811   | 5.37         | 5.37       |
| 1024 | 2     | 0.317682        | 0.014206   | 10.31        | 5.16       |
| 1024 | 4     | 0.181883        | 0.014618   | 18.01        | 4.50       |
| 1024 | 8     | <b>0.171862</b> | 0.009664   | <b>19.06</b> | 2.38       |
| 2048 | 1     | 5.588741        | 0.137242   | 6.86         | 6.86       |
| 2048 | 2     | 2.872456        | 0.047671   | 13.34        | 6.67       |
| 2048 | 4     | <b>1.859285</b> | 0.039886   | <b>20.62</b> | 5.15       |

| N    | procs | média (s)        | desvio (s) | speedup      | eficiência |
|------|-------|------------------|------------|--------------|------------|
| 2048 | 8     | 2.000130         | 0.013671   | 19.16        | 2.40       |
| 4096 | 1     | 45.167300        | 0.461076   | 7.92         | 7.92       |
| 4096 | 2     | 24.017515        | 0.171532   | 14.89        | 7.45       |
| 4096 | 4     | <b>16.708619</b> | 0.075889   | <b>21.41</b> | 5.35       |
| 4096 | 8     | 16.819975        | 0.091235   | 21.27        | 2.66       |

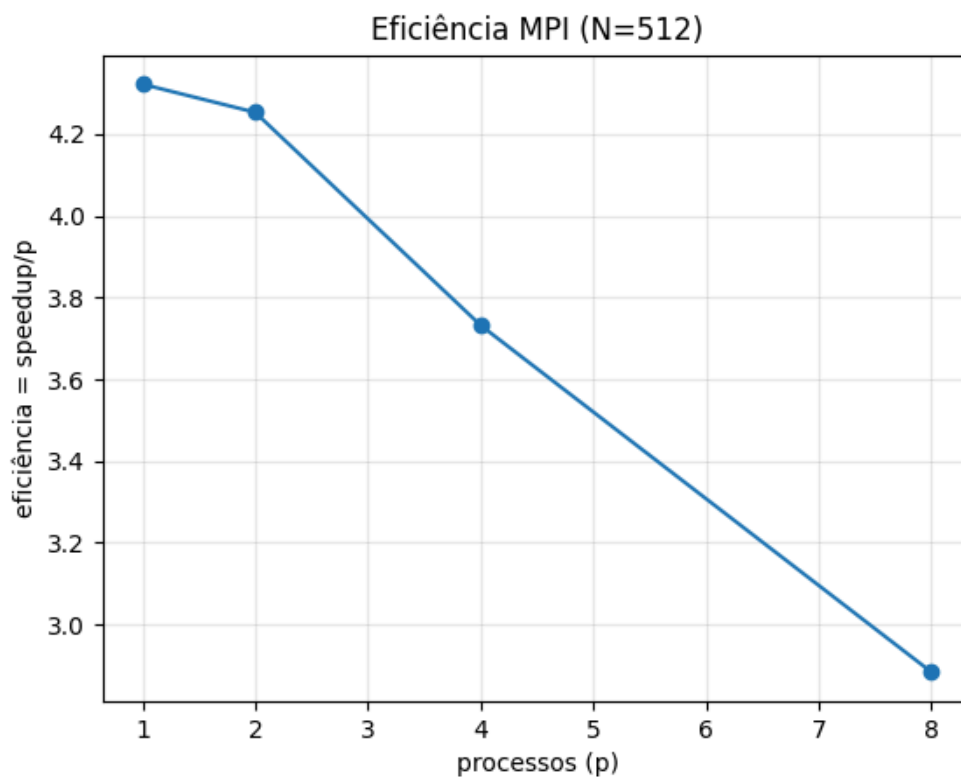
### 3.3 Gráficos

As figuras a seguir apresentam o *speedup* e a eficiência da versão MPI para cada tamanho de matriz. A base de referência é o tempo sequencial. Note que para (p=1) o *speedup* já é maior que 1 devido à otimização da transposição de (B), não ao paralelismo propriamente dito.

#### *Speedup e Eficiência para N = 512*

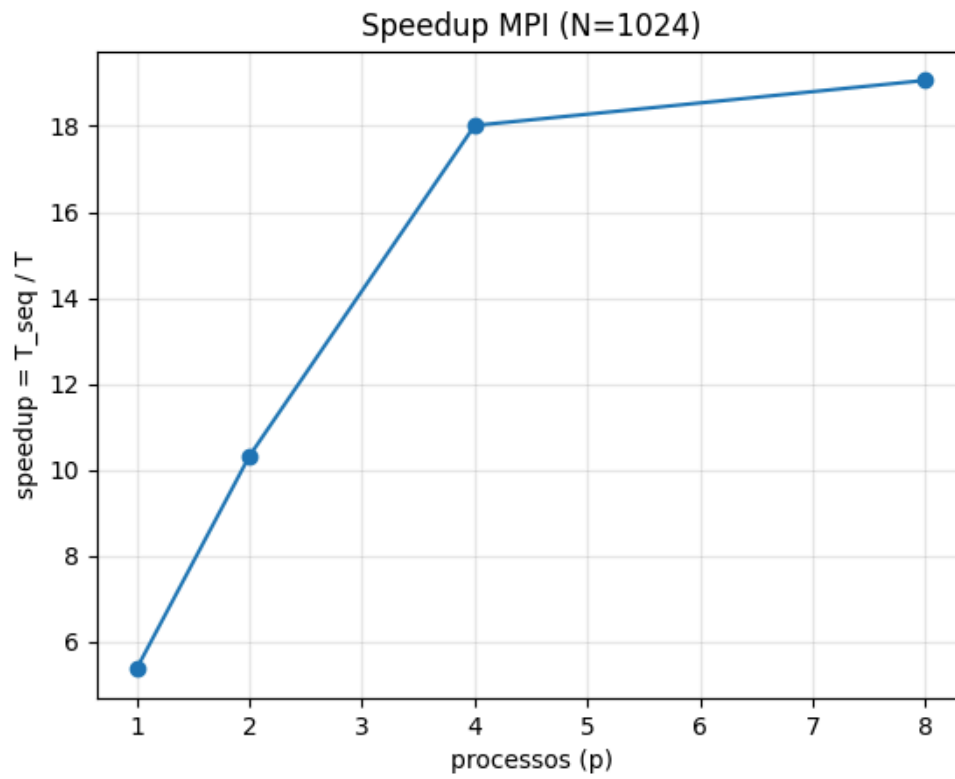


#### *Speedup MPI N=512*

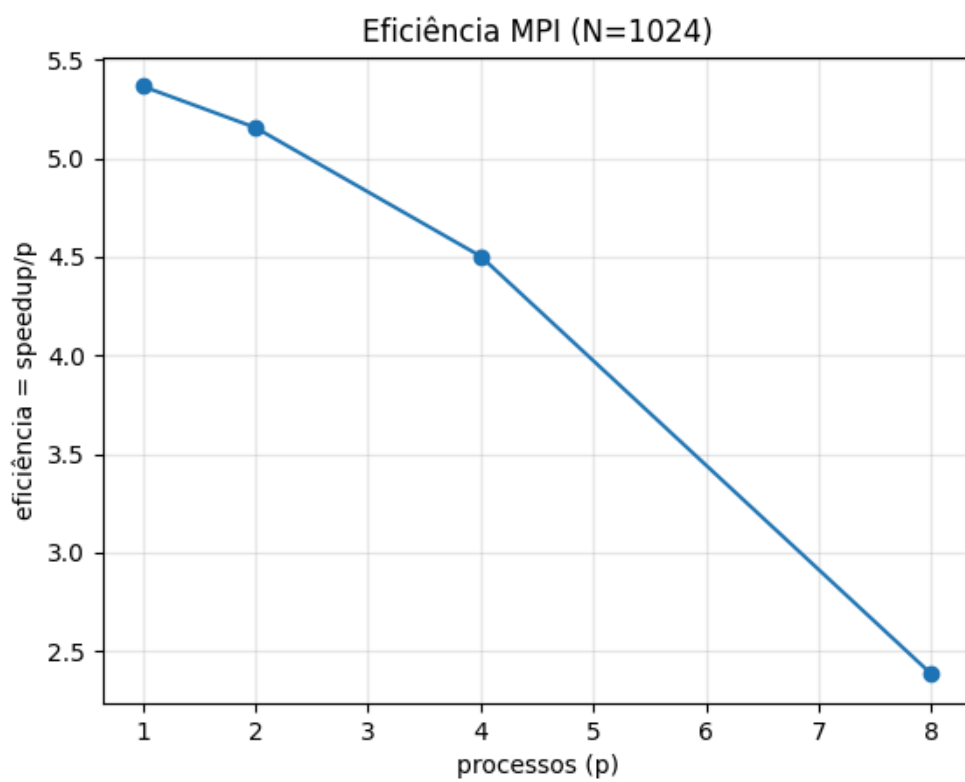


*Eficiência MPI N=512*

### *Speedup e Eficiência para $N = 1024$*



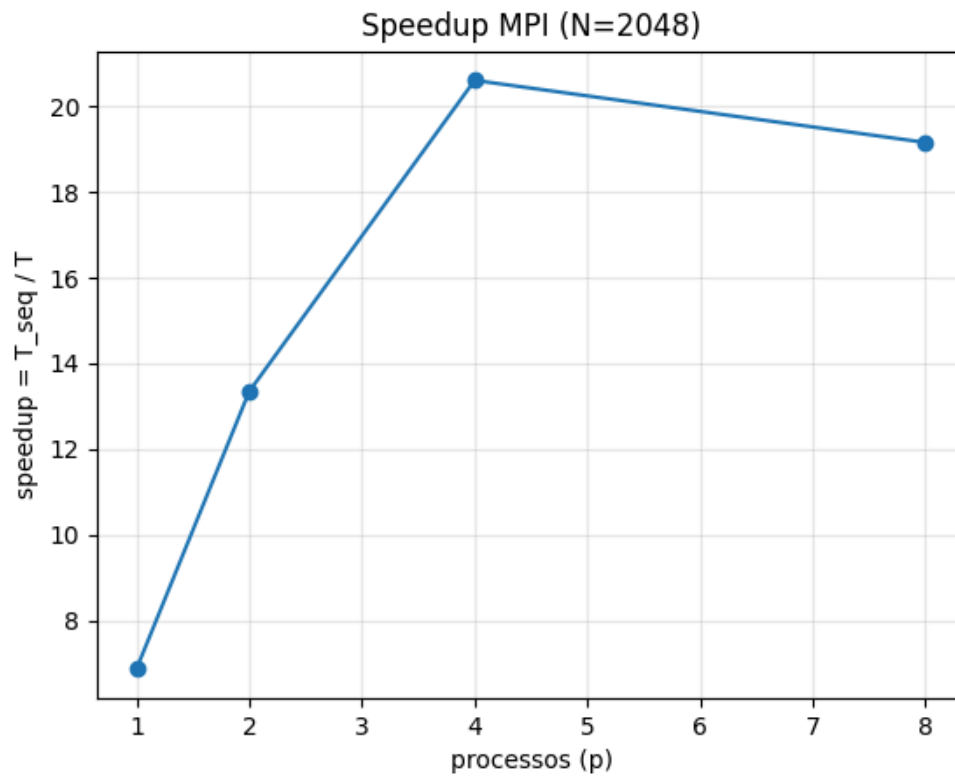
*Speedup MPI N=1024*



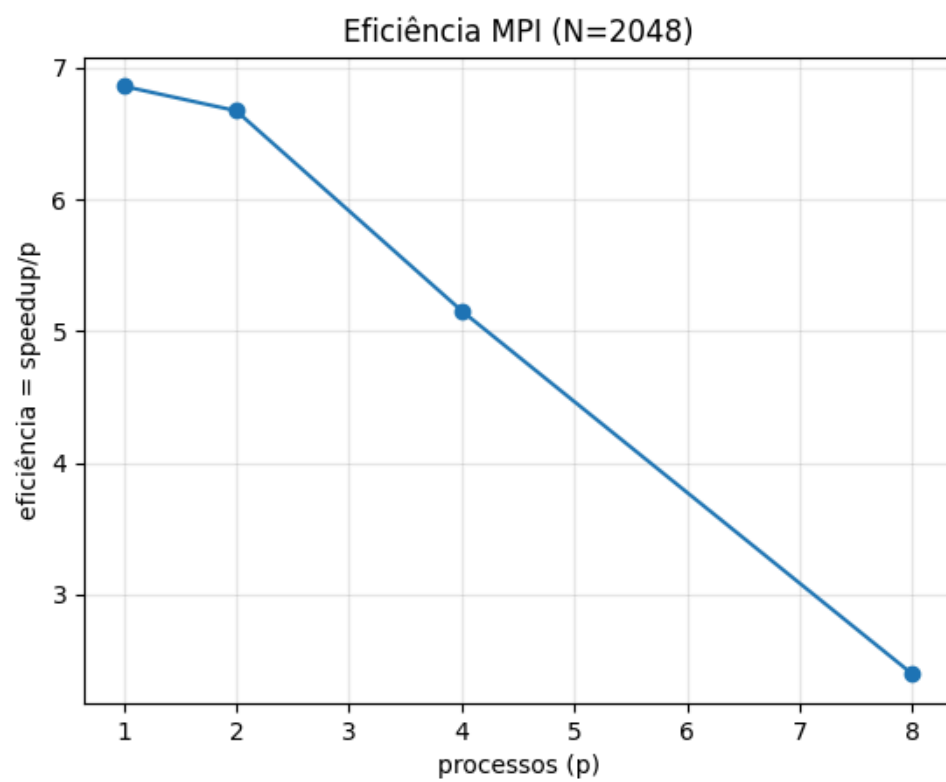
*Efficiency MPI N=1024*



### *Speedup e Eficiência para $N = 2048$*

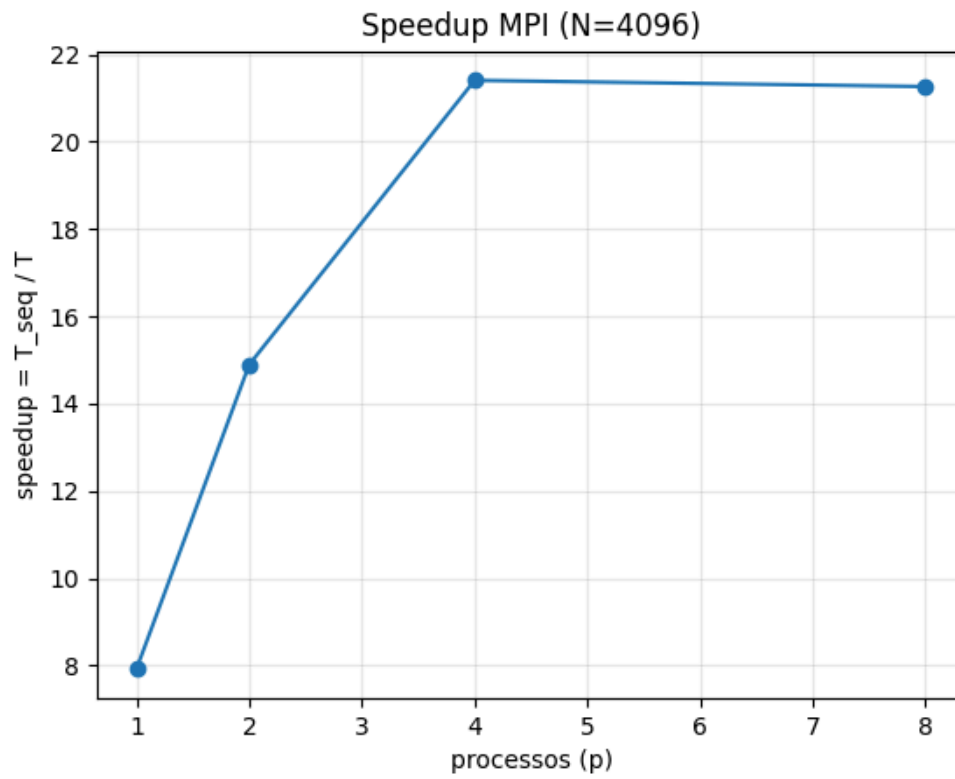


*Speedup MPI N=2048*

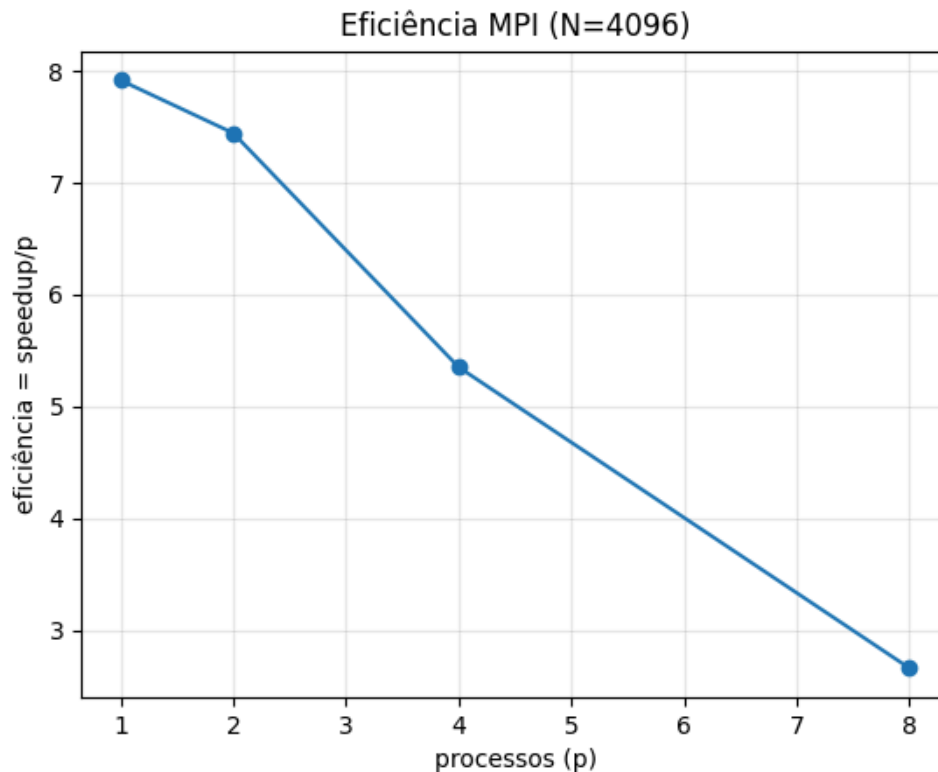


*Efficiência MPI N=2048*

### *Speedup e Eficiência para $N = 4096$*



*Speedup MPI N=4096*

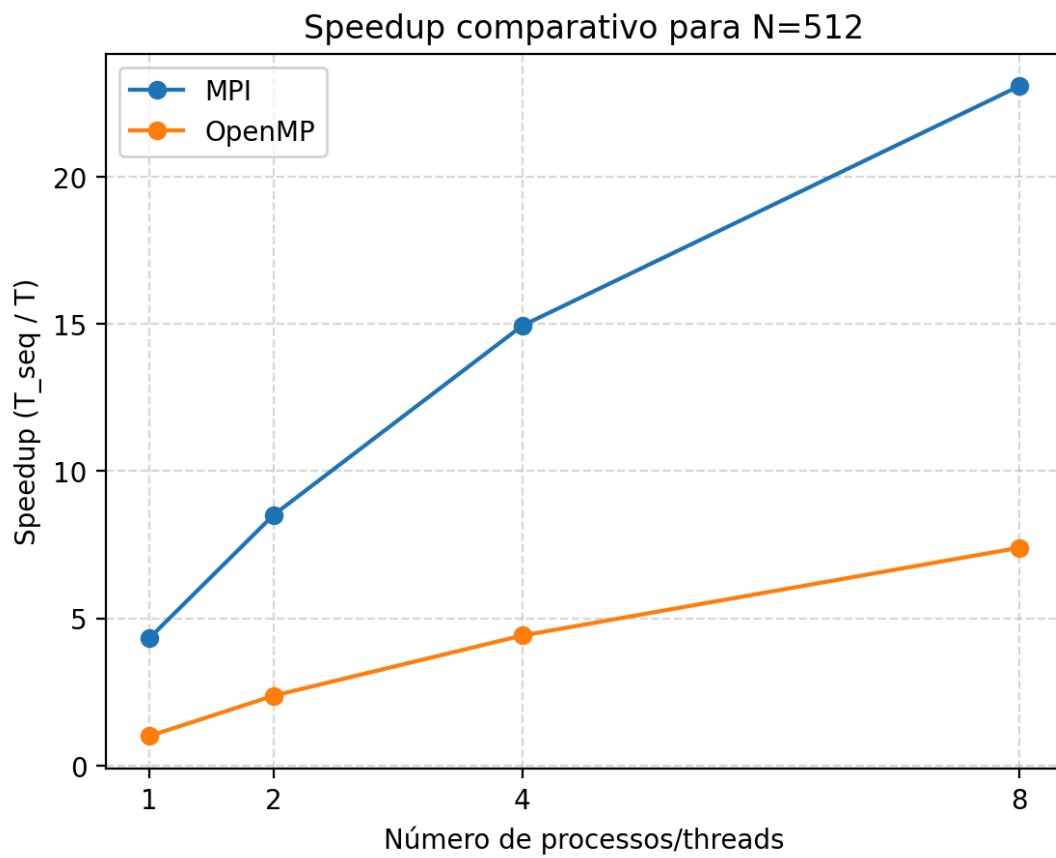


*Eficiência MPI N=4096*

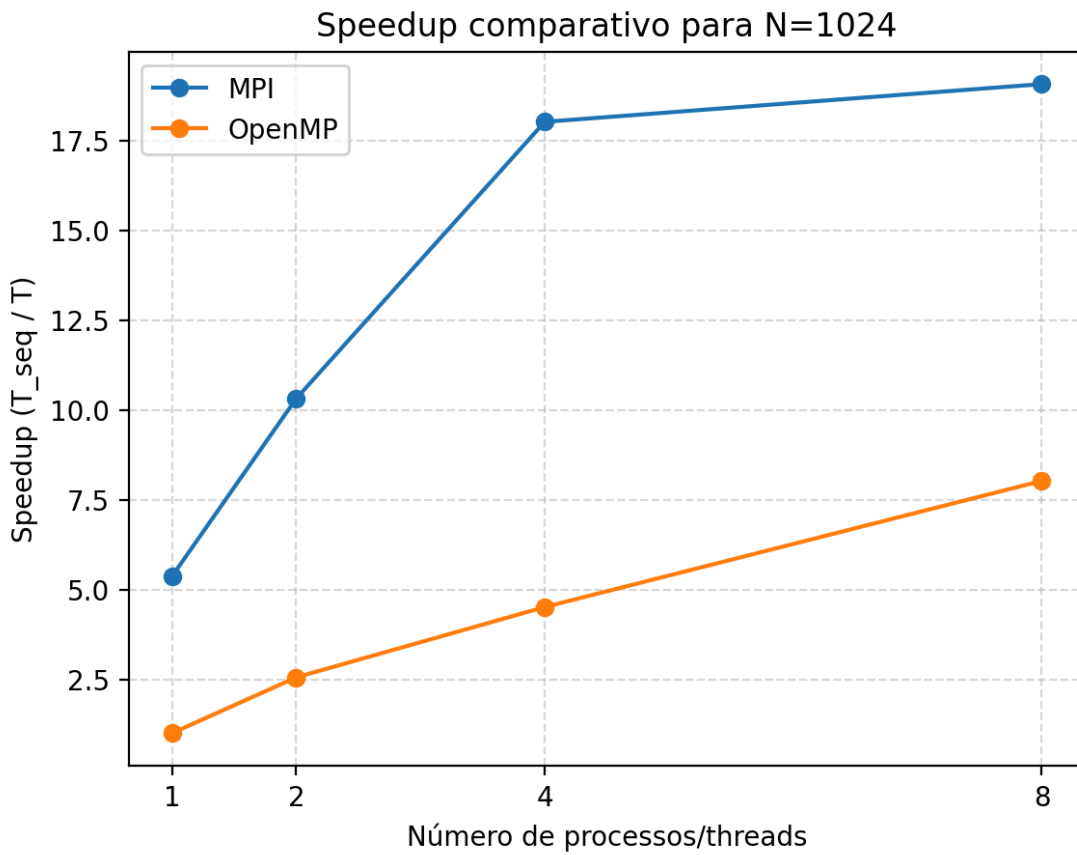
### 3.4 Análise comparativa com os resultados do Projeto 1

Uma avaliação completa de desempenho exige comparar a implementação MPI com as versões sequencial e OpenMP desenvolvidas no Projeto 1. A Tabela 2 já evidencia que, mesmo com 12 threads, o OpenMP alcançou speedup de **10,51** para (N=512) e **9,76** para (N=1024), enquanto a Tabela 3 mostra que o MPI com 8 processos atingiu *speedups* superiores a **23** e **19**, respectivamente. A figura a seguir sobrepõe as curvas de speedup de MPI e OpenMP para cada tamanho de matriz, considerando os mesmos números de processos/threads (1, 2, 4 e 8). O tempo sequencial é a referência em ambos os casos.

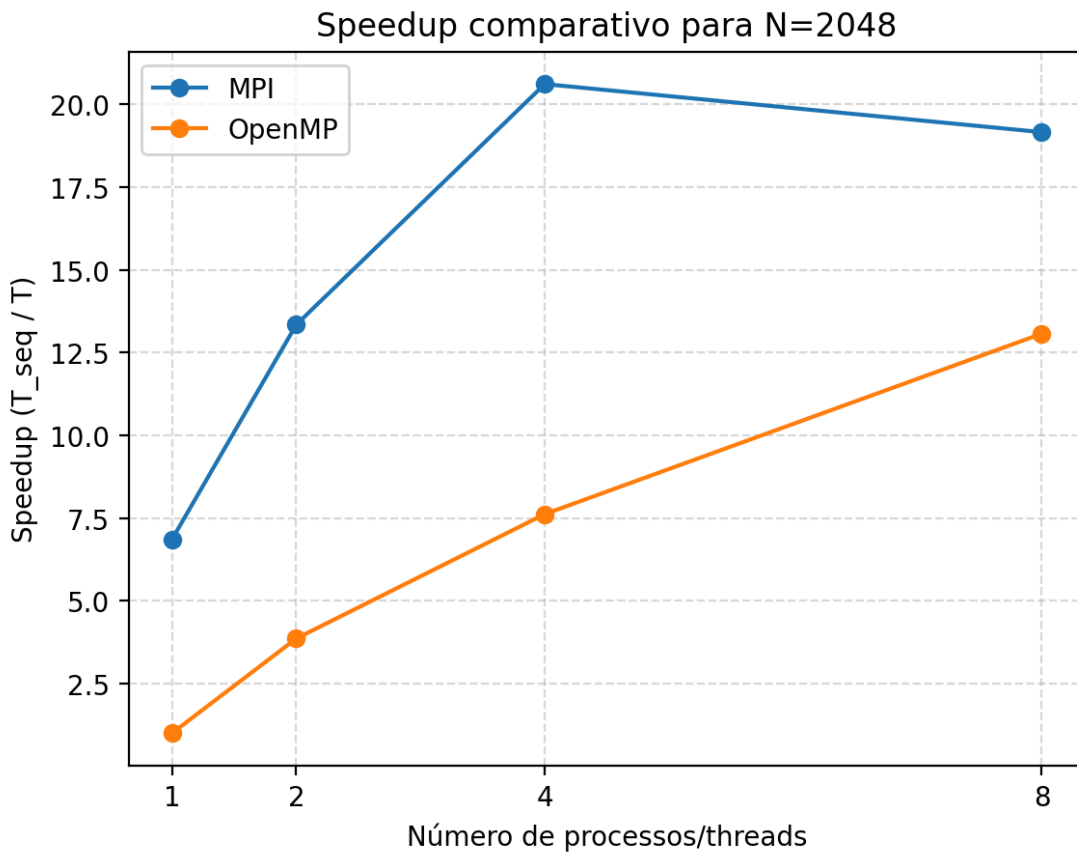
*Speedup comparativo (MPI × OpenMP) para cada N*



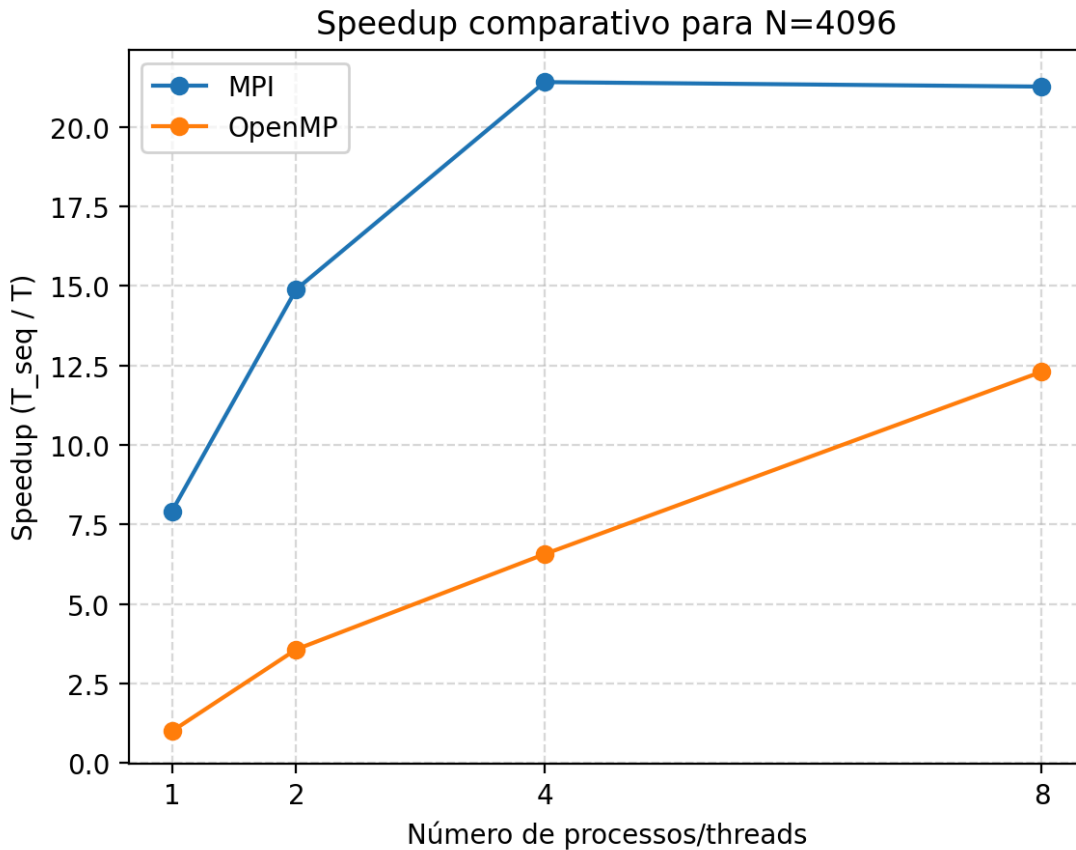
*Speedup comparativo N=512*



*Speedup comparativo N=1024*



*Speedup comparativo N=2048*



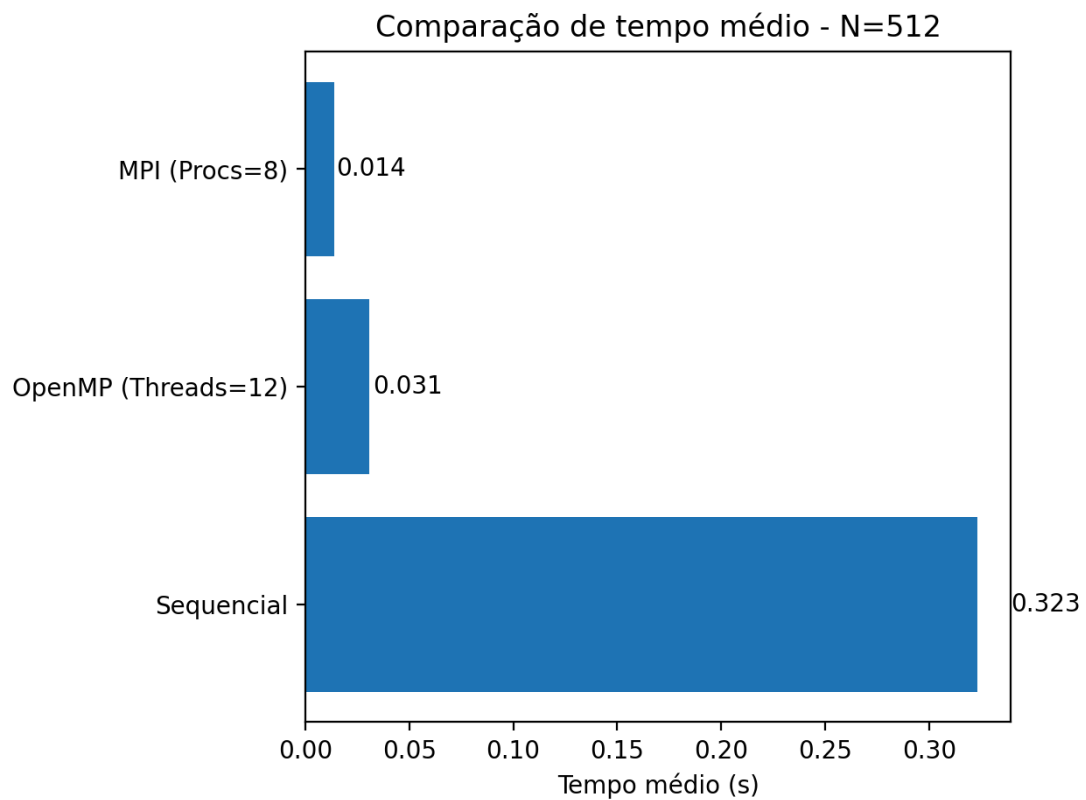
#### *Speedup comparativo N=4096*

Observa-se que a versão MPI domina a OpenMP em todos os tamanhos e números de processos testados. A diferença é mais pronunciada para (N=512) e (N=1024), onde o OpenMP apresenta ganho modesto além de 8 threads devido à saturação da memória compartilhada. Para (N=2048) e (N=4096), a computação domina a comunicação, e o MPI obtém *speedups* de até 21, enquanto o OpenMP se estabiliza em torno de 7.

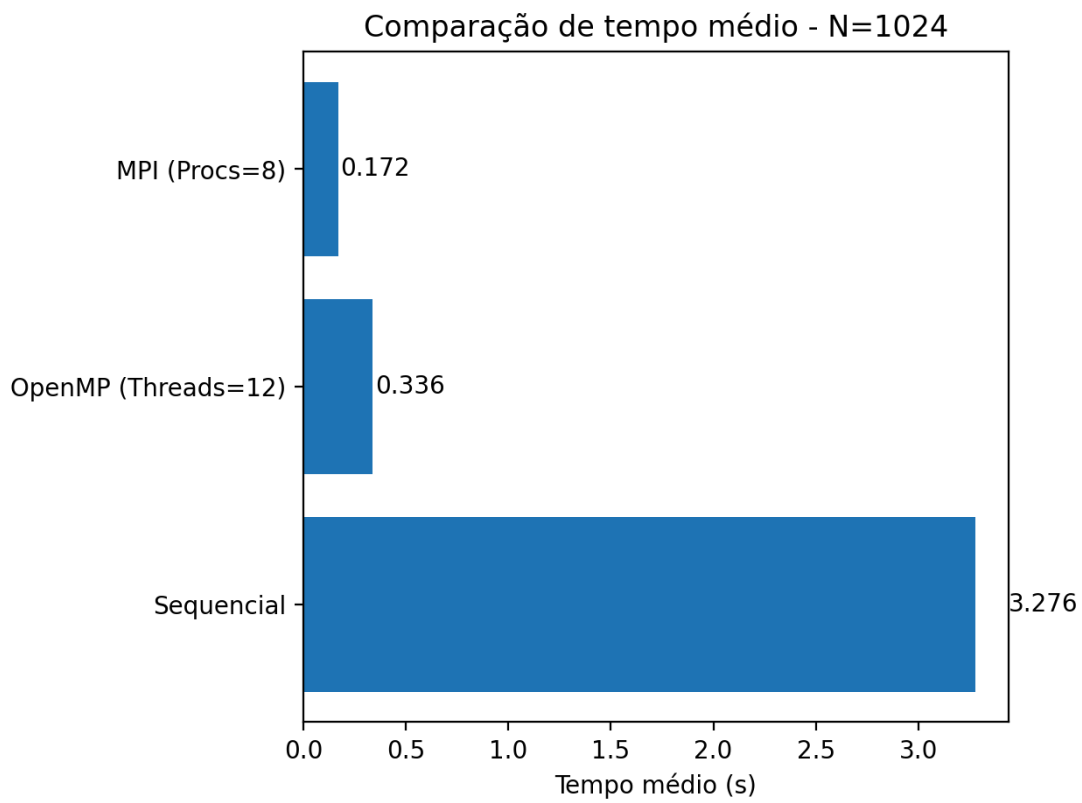
### 3.5 Gráficos de barras horizontais (tempo médio)

Para visualizar de maneira intuitiva o impacto das otimizações, apresenta-se a comparação do tempo médio da versão sequencial, da melhor configuração OpenMP e da melhor configuração MPI para cada (N). Esses gráficos de barras horizontais evidenciam a magnitude da redução de tempo obtida com a paralelização e com a otimização algorítmica da transposição de (B).

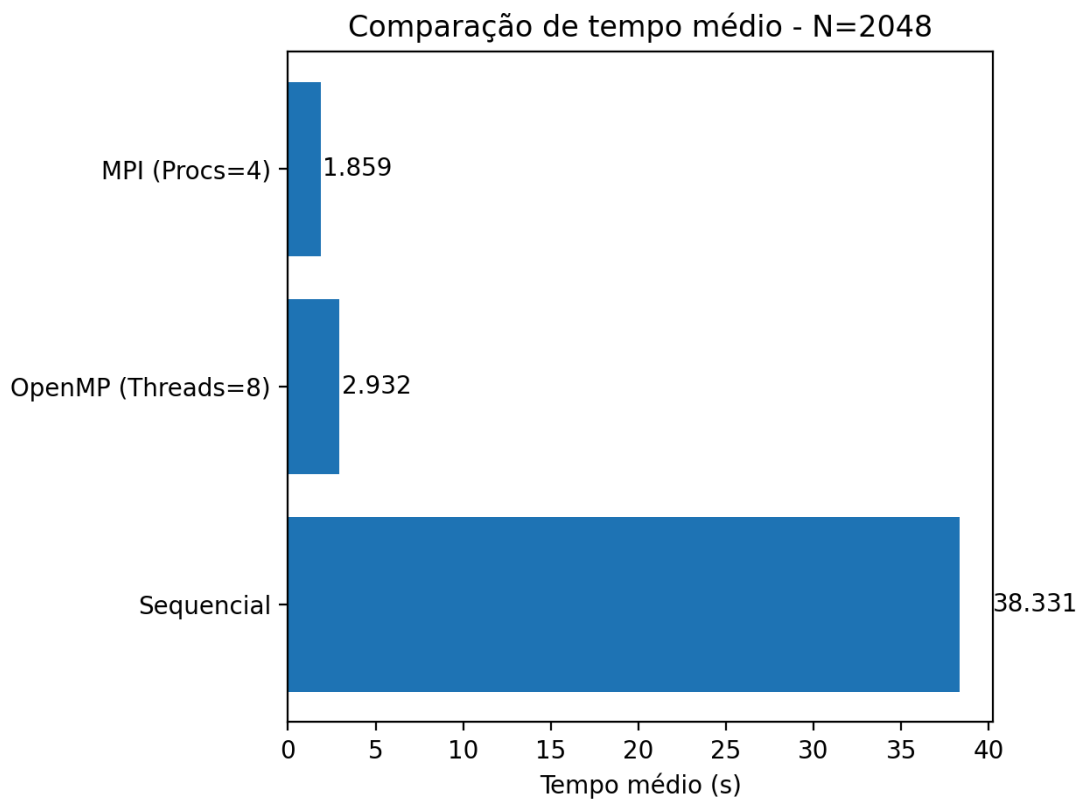




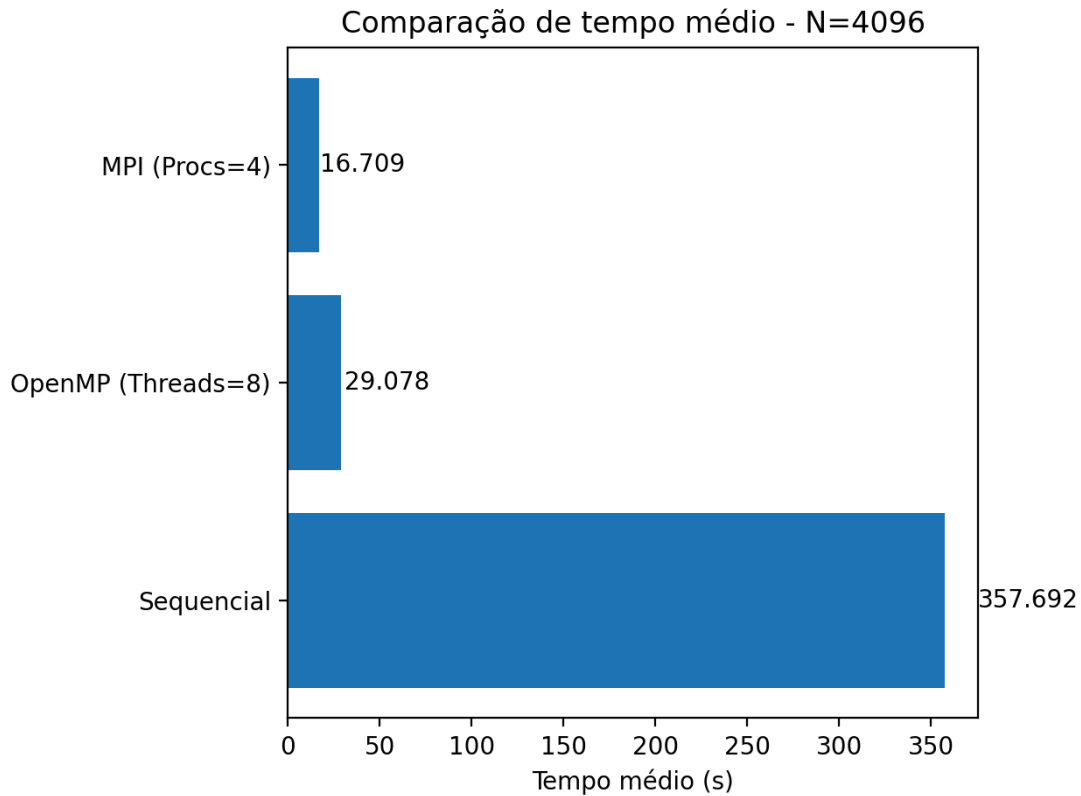
*Comparação de tempo N=512*



*Comparação de tempo N=1024*



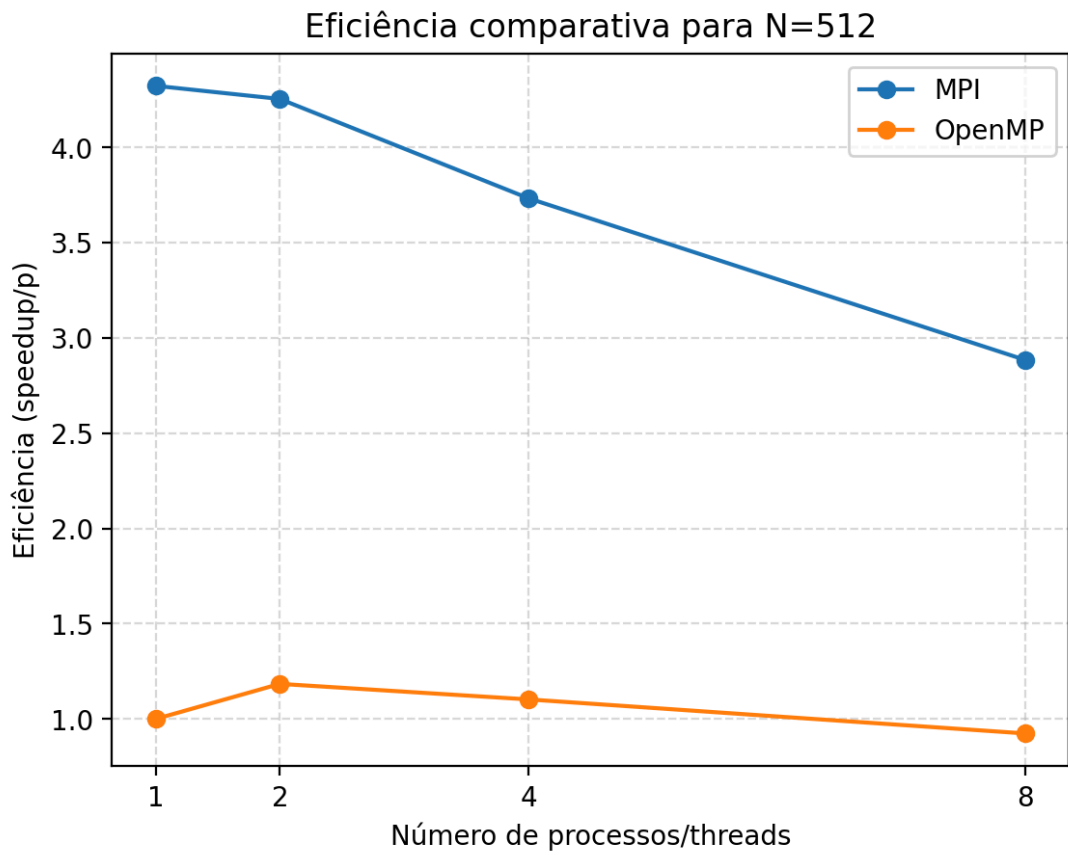
*Comparação de tempo N=2048*



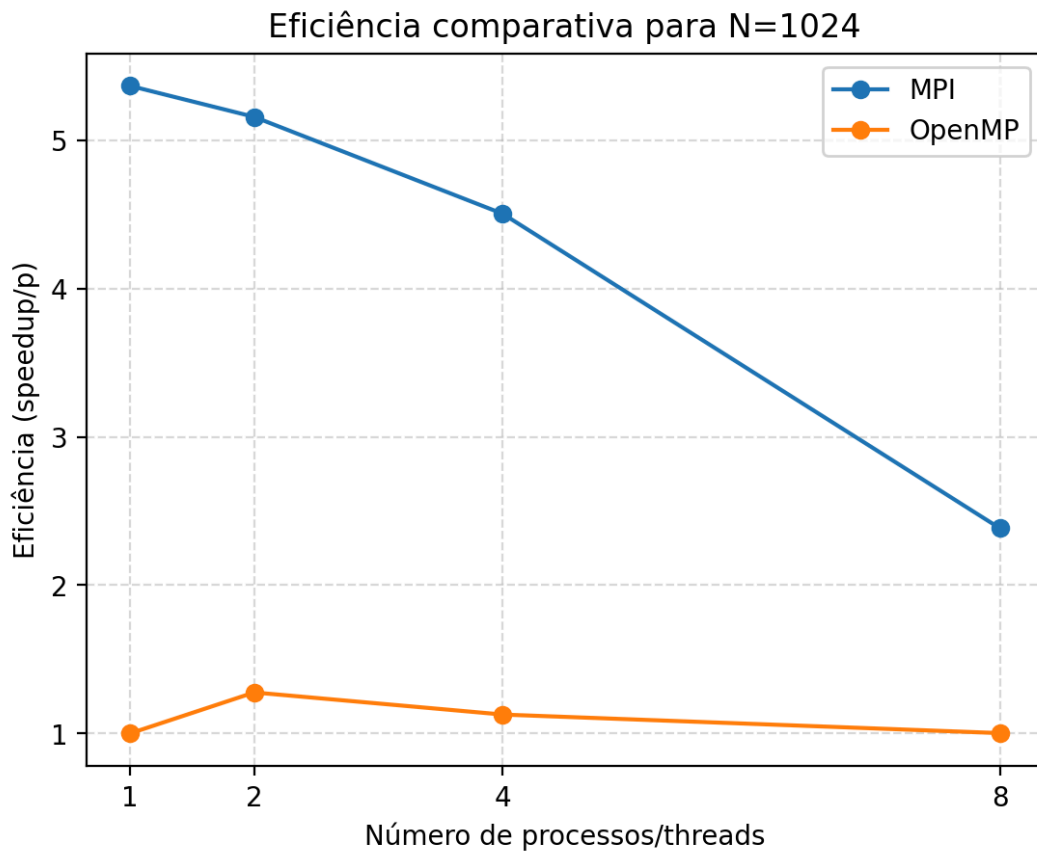
*Comparação de tempo N=4096*

### 3.6 Comparação de eficiência (MPI × OpenMP)

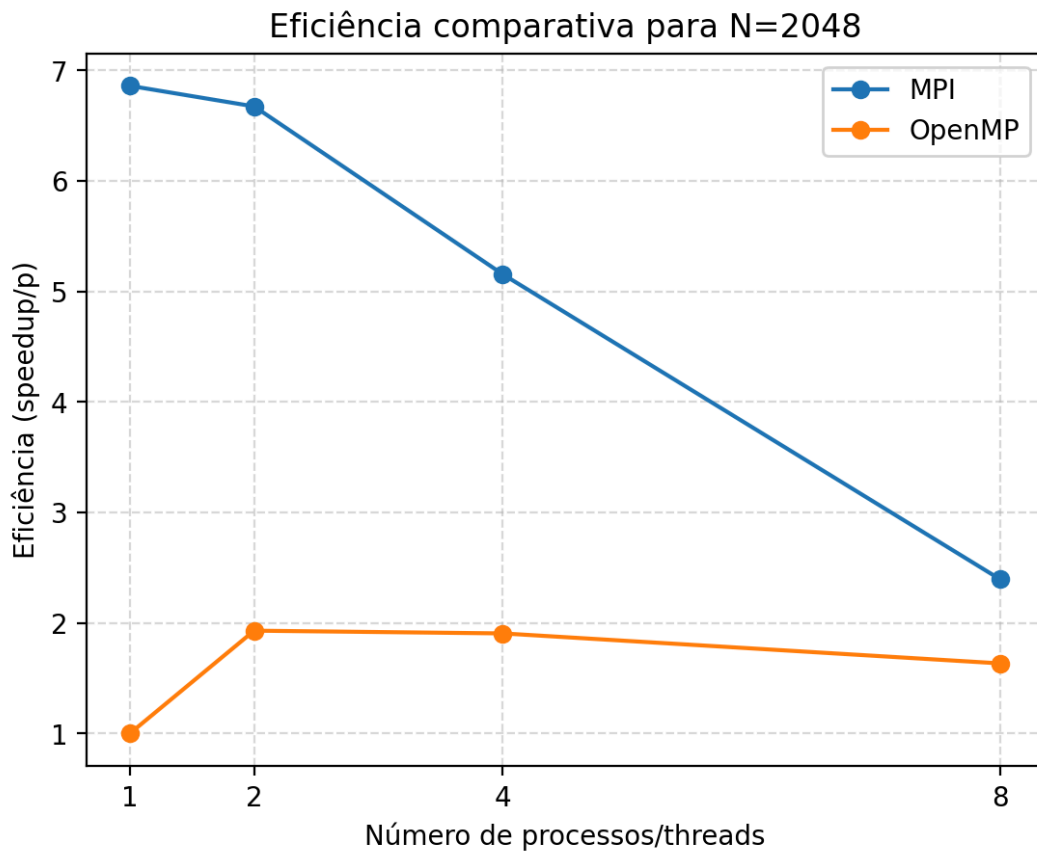
Para observar como a eficiência varia com o número de processos/threads, apresentam-se os gráficos de eficiência comparativa entre as versões MPI e OpenMP para cada tamanho de matriz. A eficiência é calculada como o *speedup* dividido pelo número de processos ou threads.



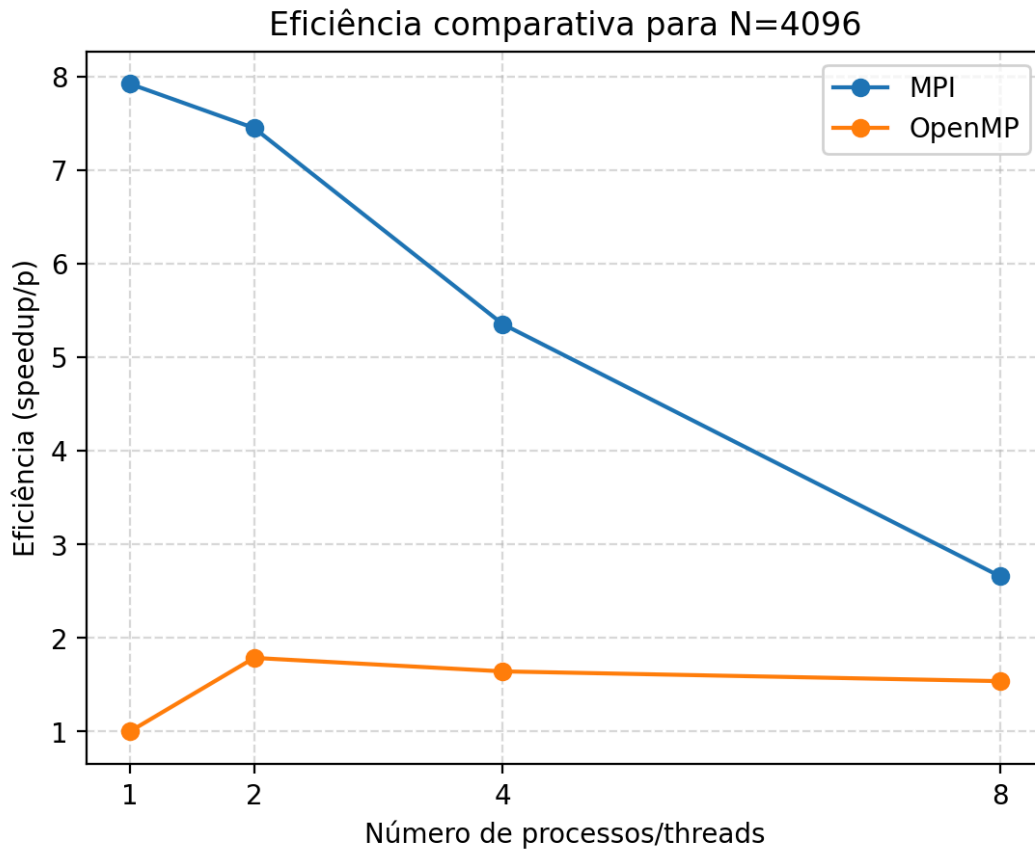
*Eficiência comparativa N=512*



*Eficiência comparativa N=1024*



*Eficiência comparativa N=2048*

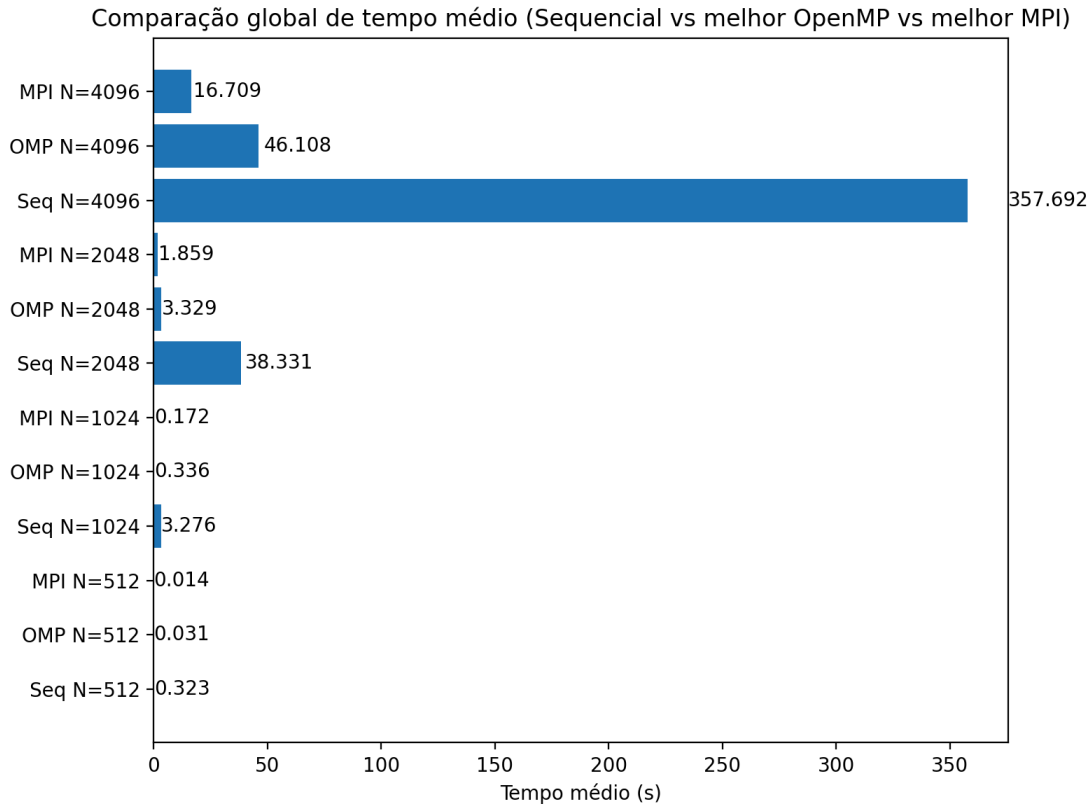


*Eficiência comparativa N=4096*

### 3.7 Comparação global de tempo médio

Os tempos médios dos três modelos (sequencial, melhor OpenMP e melhor MPI) foram reunidos em um único gráfico de barras horizontais. Essa figura permite comparar diretamente o ganho absoluto de tempo para cada tamanho de matriz e evidenciar a superioridade da versão MPI quando o objetivo é reduzir o tempo de execução.





### Comparação global de tempos

## 4. Discussão

Os resultados indicam que a versão MPI alcança *speedups* significativos em relação à implementação sequencial **e que supera de forma consistente** os ganhos obtidos com a versão OpenMP do Projeto 1. Conforme mostrado nos gráficos comparativos da Seção 3.4, para os mesmos números de processos/threads (1, 2, 4 e 8) o MPI oferece *speedups* muito maiores: em (N=512), a melhor configuração OpenMP (8 threads) obteve 7,39 de speedup, ao passo que o MPI com 8 processos atingiu 23,07; em (N=1024) a OpenMP alcançou 8,02 com 8 threads, enquanto o MPI chegou a 19,06. Para (N=512), o speedup de MPI cresce de 4,32 (p=1) para 8,51 (p=2) e 14,93 (p=4), enquanto em (p=8) o ganho adicional diminui devido ao overhead de comunicação e replicação de dados. A eficiência cai de 4,25 em (p=2) para 2,88 em (p=8), evidenciando perdas de escalabilidade quando o número de processos duplica. Em (N=1024), o melhor trade-off ocorre em (p=4), atingindo speedup de 18,01 e eficiência de 4,50; para (p=8), o ganho marginal é reduzido porque as mensagens e o acesso à memória tornam-se gargalos.

Para matrizes maiores, (N=2048), a computação domina a comunicação, permitindo que (p=4) alcance speedup de 20,62. No entanto, (p=8) apresenta queda de eficiência (2,40), possivelmente porque o ambiente WSL 2 não dispõe de memória suficiente para armazenar todas as cópias de (B) e (B<sup>^</sup>) sem recorrer a swap. Em (N=4096), tanto

( $p=4$ ) quanto ( $p=8$ ) obtêm speedups superiores a 21, mas a eficiência cai para 2,66 em ( $p=8$ ); a largura de banda da memória torna-se o gargalo principal, e a replicação de (B) em cada processo limita a escalabilidade.

Comparando com a versão OpenMP, nota-se que a eficiência com threads é mais baixa: o melhor speedup para ( $N=4096$ ) foi 7,76 com 12 threads, refletindo saturação da memória compartilhada. A versão MPI, por dividir (A) entre processos e transpor (B), reduz o volume de dados acessados por cada processo e alcança speedups superiores. Contudo, deve-se lembrar que a transposição de (B) já acelera o caso ( $p=1$ ); uma comparação justa requereria otimizar igualmente a versão sequencial. Além disso, as bibliotecas de comunicação em clusters reais possuem interconexões mais rápidas; os resultados no WSL 2 podem não refletir a escalabilidade real em um supercomputador.

## 5. Conclusão

O Projeto 2 demonstrou que a multiplicação de matrizes DGEMM pode ser eficientemente paralelizada em um ambiente de memória distribuída utilizando MPI. A decomposição por blocos de linhas combinada com a transposição de (B) possibilitou a obtenção de *speedups* superiores a 20 para ( $N=2048$ ) e ( $N=4096$ ) com quatro processos, validando a eficácia do modelo distribuído. A verificação de corretude confirmou que a diferença relativa máxima entre a solução MPI e a sequencial é praticamente nula. Comparando com o OpenMP, a versão MPI se mostrou mais escalável no hardware disponível, embora sua eficiência diminua à medida que o número de processos cresce devido ao overhead de comunicação e às limitações de memória no WSL 2.

Trabalhos futuros incluem otimizar a versão sequencial com transposição para uma comparação mais justa, testar o programa em clusters reais com rede de alta velocidade, explorar blocagens internas e combinar MPI com OpenMP (modelo híbrido). Essas abordagens podem melhorar a eficiência e reduzir o impacto da comunicação.